

Merchant Customer Loyalty Platform - MVP Requirements

Version: 1.0 Status: Draft Document Type: Product Requirements

1. Introduction

1.1 Vision

Build a lightweight Merchant-Customer engagement platform where merchants and customers can establish a relationship using QR codes and continue the interaction through feedback threads.

The MVP is intentionally limited to registration, authentication, QR-based association, and feedback conversations. The platform should still be designed so future capabilities like promotions, loyalty points, notifications, and LLM summaries can be added without rewriting the core model.

1.2 Goals

- Allow merchants to self-register as individual store locations.
- Allow customers to self-register using phone number and name.
- Authenticate both actors using SMS OTP and issue JWT tokens for secured APIs.
- Allow a customer to scan a merchant QR code and become associated with that merchant.
- Allow customers to create multiple feedback threads for the same merchant.
- Allow merchants to reply to feedback and close feedback threads.
- Expose REST APIs that can later be consumed by web, mobile, or external systems.

1.3 MVP Assumptions

- Merchant represents one physical store/location. Store hierarchy is future scope.
- A customer may be associated with many merchants. A merchant may have many customers.
- Association is permanent in MVP. Removing or opting out is future scope.
- SMS OTP is the only login method for MVP.
- QR codes are permanent and generated once.
- Feedback comments are text only. No images, files, reactions, edits, or deletes in MVP.
- The first implementation can be tested using Postman JSON payloads before Angular UI is built.

2. Business Context

Small merchants often receive feedback through disconnected channels such as verbal conversations, social media, reviews, email, and text messages. This platform centralizes feedback into a direct merchant-customer relationship.

QR codes reduce onboarding friction. A merchant can display a QR code at the store, and a customer can scan it to start a relationship and immediately create feedback. This simulates real-world product behavior while keeping the first version simple.

3. Actors

3.1 Customer

A customer is an authenticated user who interacts with one or more merchants. The customer is identified primarily by phone number.

- Register using name and phone number.
- Login using SMS OTP.
- Receive JWT after OTP verification.
- Scan merchant QR code.
- Become associated with merchant.

- Create feedback thread.
- View own feedback threads.
- Reply to feedback thread.

3.2 Merchant

A merchant represents a single physical store location. Each merchant has its own account, QR code, customers, and feedback threads.

- Register store using basic details.
- Login using SMS OTP.
- Receive JWT after OTP verification.
- Display merchant QR code.
- View associated customers.
- View feedback threads created for that merchant.
- Reply to feedback threads.
- Close feedback threads.

4. Authentication and Authorization

4.1 Authentication Method

SMS OTP is used for both customer and merchant login. Once the OTP is verified, the Auth Service generates a JWT token. The JWT token is required for all secured APIs.

4.2 Login Flow

- User enters phone number.
- System sends OTP to the phone number.
- User enters OTP.
- Auth Service validates OTP.
- Auth Service generates JWT.
- Client sends JWT in Authorization header for all subsequent API calls.

4.3 Authorization Header

Authorization: Bearer <jwt>

4.4 JWT Claims

Claim	Description
sub	User identifier
role	CUSTOMER or MERCHANT
exp	Token expiration time

4.5 Access Rules

- Customer can access only own profile, associated merchants, and own feedback threads.
- Merchant can access only own profile, associated customers, and feedback threads belonging to that merchant.

5. Merchant Requirements

5.1 Definition

Merchant equals a single store/location for MVP. Example: ABC Pizza - Fremont and ABC Pizza - San Jose are treated as two separate merchants.

5.2 Functional Requirements

- Merchant can self-register.
- Merchant registration requires store name, phone number, address, and category.
- Merchant email is optional.

- Merchant authenticates using SMS OTP.
- Merchant receives JWT after login.
- Merchant has a permanent QR code.
- Merchant can view associated customers.
- Merchant can view, reply to, and close feedback threads.

5.3 Merchant Categories

- RESTAURANT
- GROCERY
- SALON
- RETAIL
- OTHER

5.4 Merchant Table

Column	Type	Nullable	Description
merchant_id	UUID	No	Primary key
store_name	VARCHAR(100)	No	Store name
phone_number	VARCHAR(20)	No	Unique phone number
email	VARCHAR(100)	Yes	Optional email
address	VARCHAR(255)	No	Physical address
category	VARCHAR(30)	No	Merchant category
qr_code	VARCHAR(100)	No	Unique QR identifier
created_date	TIMESTAMP	No	Creation timestamp

6. Customer Requirements

6.1 Definition

Customer is an end user who can associate with merchants and provide feedback.

6.2 Functional Requirements

- Customer can self-register.
- Customer registration requires phone number and name.
- Email is optional.
- Customer authenticates using SMS OTP.
- Customer receives JWT after login.
- Customer has a permanent QR code.
- Customer can scan merchant QR code.
- Customer can create and view own feedback threads.

6.3 Customer Table

Column	Type	Nullable	Description
customer_id	UUID	No	Primary key
phone_number	VARCHAR(20)	No	Unique phone number
name	VARCHAR(100)	No	Customer name
email	VARCHAR(100)	Yes	Optional email
qr_code	VARCHAR(100)	No	Unique QR identifier
created_date	TIMESTAMP	No	Creation timestamp

7. Merchant Customer Association

7.1 Purpose

The association table represents the relationship between a merchant and customer. This is the foundation for future loyalty, promotions, and customer engagement features.

7.2 Business Rules

- A customer can be associated with multiple merchants.
- A merchant can be associated with multiple customers.
- Association is created when customer scans merchant QR.
- Duplicate association is not allowed.
- Association removal is not supported in MVP.

7.3 Customer Scans Merchant QR Flow

- Customer scans merchant QR code.
- System identifies merchant from QR code.
- System checks if customer and merchant are already associated.
- If not associated, system creates association.
- If already associated, customer can create feedback or view previous feedback.

7.4 Association Table

Column	Type	Nullable	Description
association_id	UUID	No	Primary key
merchant_id	UUID	No	Foreign key to merchant
customer_id	UUID	No	Foreign key to customer
created_date	TIMESTAMP	No	Association timestamp

Constraint: UNIQUE(merchant_id, customer_id)

8. QR Code Requirements

8.1 Merchant QR

- Permanent for MVP.
- Generated once during merchant registration.
- Uniquely identifies merchant.
- Displayed by merchant for customer scanning.

8.2 Customer QR

- Permanent for MVP.
- Generated once during customer registration.
- Uniquely identifies customer.
- Stored for future merchant-scan use cases, but merchant scanning customer QR is not in MVP.

8.3 Storage

Only the QR identifier/string is stored. The QR image can be generated dynamically by the application when needed.

9. Feedback Management

9.1 Purpose

Feedback threads allow customers and merchants to have a focused conversation about a specific service, transaction, or experience.

9.2 Business Rules

- Customer can create multiple feedback threads for the same merchant.
- Each thread is independent.
- Thread starts with OPEN status.
- Merchant can reply to the thread.
- Merchant can close the thread.
- Closed thread cannot be reopened in MVP.
- Customer can create a new thread for future interactions.

9.3 Thread Status

- OPEN - active conversation.
- CLOSED - resolved or completed conversation.

9.4 Feedback Thread Table

Column	Type	Nullable	Description
thread_id	UUID	No	Primary key
merchant_id	UUID	No	Foreign key to merchant
customer_id	UUID	No	Foreign key to customer
subject	VARCHAR(255)	No	Short thread subject
status	VARCHAR(20)	No	OPEN or CLOSED
created_date	TIMESTAMP	No	Creation timestamp
last_updated_date	TIMESTAMP	No	Last update timestamp

10. Comment Requirements

10.1 Purpose

Comments represent the actual conversation messages inside a feedback thread.

10.2 Business Rules

- Customer and merchant can add comments to a thread.
- Comments are text only.
- Comments are immutable in MVP.
- Edit and delete are not supported.
- Images, attachments, rich text, and reactions are not supported.

10.3 Comment Table

Column	Type	Nullable	Description
comment_id	UUID	No	Primary key
thread_id	UUID	No	Foreign key to feedback thread
created_by	VARCHAR(20)	No	CUSTOMER or MERCHANT
message	TEXT	No	Comment text
created_date	TIMESTAMP	No	Creation timestamp

11. User Journeys

11.1 Merchant Registration

- Merchant enters store name, phone number, address, and category.
- Merchant verifies phone number using OTP.
- Merchant account is created.
- Merchant QR code is generated.

11.2 Customer Registration

- Customer enters name and phone number.
- Customer verifies phone number using OTP.
- Customer account is created.
- Customer QR code is generated.

11.3 Customer Scans Merchant QR

- Customer scans merchant QR.
- System validates merchant QR.
- System creates association if missing.
- Customer can create feedback or view previous feedback.

11.4 Customer Creates Feedback

- Customer selects associated merchant.
- Customer enters subject and first comment.
- System creates feedback thread with OPEN status.
- System stores first comment.

11.5 Merchant Replies

- Merchant views feedback threads.
- Merchant opens a thread.
- Merchant adds text reply.
- System updates last_updated_date.

11.6 Merchant Closes Thread

- Merchant selects OPEN thread.
- Merchant changes status to CLOSED.
- Thread becomes closed for MVP.

12. High-Level API Scope

Area	Example APIs
Auth	POST /auth/send-otp, POST /auth/verify-otp
Merchant	POST /merchants, GET /merchants/me, GET /merchants/me/qr
Customer	POST /customers, GET /customers/me, GET /customers/me/qr
Association	POST /associations/scan-merchant, GET /customers/me/merchants
Feedback	POST /feedback-threads, GET /feedback-threads, PATCH /feedback-threads/{id}/close
Comments	POST /feedback-threads/{id}/comments, GET /feedback-threads/{id}/comments

13. Microservices

13.1 Auth Service

- Generate OTP.
- Validate OTP.
- Generate JWT.
- Validate JWT for secured APIs.

13.2 Business Service

- Merchant registration and profile.
- Customer registration and profile.
- QR code identifier management.
- Merchant-Customer association.
- Feedback threads.
- Comments.

14. Non-Functional Requirements

- REST APIs should be used for all external interactions.
- Services should be stateless where possible.
- JWT should protect secured APIs.
- HTTPS should be used in deployed environments.
- Relational database should be used for MVP.

- Basic logging and health checks are required.
- Correlation ID support should be added for request tracing.
- Services should be deployable independently later, even if initially implemented simply.

15. Explicitly Out of Scope

- Admin portal.
- Merchant hierarchy and parent-child merchants.
- Merchant scans customer QR.
- Association removal and customer opt-out.
- Promotions, coupons, rewards, and loyalty points.
- Notifications by SMS, email, or push.
- LLM summaries and business insights.
- Images, attachments, comment edits, and comment deletes.
- Analytics dashboard.
- Rate limiting and DDoS protection.
- Social login and email/password login.